BEACON LABS

CYBERSECURITY EDUCATION LABS

Android Physical Input Hijacking (PHYjacking) Attack Lab

Module: Mobile Security

BOISE STATE UNIVERSITY

COLLEGE OF ENGINEERING

Department of Computer Science

Table of Contents

Overview

Objectives

Understand conventional attack strategies on mobile devices and some of the vulnerabilities within Android API's, explore the Android Activity Lifecycle and methodologies used to exploit its vulnerabilities, and implement a delusive overlay that navigates a user to a target destination and garner unauthorized access to a wire transfer

Nowadays, most mobile devices are equipped with various hardware interfaces such as touchscreen, fingerprint scanner, camera and microphone to capture inputs from the user. Many mobile apps use these physical interfaces to receive user-input for authentication/authorization operations including one-click login, fingerprint-based payment approval, and face/voice unlocking. Our focus on this lab will be to mislead an unsuspecting user with a zero-permission malicious application to feed screen-touch and fingerprint input in order to unintentionally authorize a fund-transfer. This lab is based off of the paper PHYjacking: Physical Input Hijacking for Zero-Permission Authorization Attacks on Android by X. Wang, S. Shi, Y. Chen, and W. Lau [1].

Reading

This lab is based on a paper by Xianbo Wang, Shangcheng Shi, Yikang Chen, and Wing Cheung Lau [1], and was published in the Network and Distributed System Security Symposium. Reading the original paper is not necessary but can help if you would like to learn more about this attack.

X. Wang, S. Shi, Y. Chen, W. Lau, "PHYjacking: Physical Input Hijacking for Zero-Permission Authorization Attacks on Android", 2021 [1].

Setup

Download and install the Android Studio IDE.

A provided zipped folder contains all necessary files in order to run and complete this lab. It is recommended that you have 8 GB RAM or more, x86_64 CPU architecture; 2nd generation Intel Core or newer, or AMD CPU with support for a Windows Hypervisor / ARM-based chips, or 2nd generation Intel Core or newer with support for Hypervisor.Framework, and 9 GB of available disk space minimum or more.

First create your emulated device. With the provided project open, on the right hand side of your screen, click the 'Device Manager' tab, and select 'Create device'. Scroll down and select the 'Pixel 5' option.

When choosing your system image, select the second tab that corresponds to your CPU architecture (x86 / ARM). Scroll down and select API 28 for Android 9 Pie. Make sure your application binary interface corresponds to your matching architecture. Lastly, make sure beneath the target column that your system image does not utilize Google API's or services.

Now that you have created your device, run the device to confirm that it does so successfully. If any issues occur, refer to the Troubleshooting section at the end of this document or consult with Google. You will notice your device will be in a factory-stock state with the default applications installed.

In order to install the necessary applications for this lab, locate the project manager tab on the left side of your IDE. You will have two modules, 'app' and 'raceattack'. Navigate through the hierarchy until you locate the MainActivity.java file. Right click this and select 'Run 'MainActivity.java'' for each module. You will notice the emulator launches and your application is installed. This will be how you compile your changes to the code. Utilize Figure 1 on page 5 below for an example.

Lastly, locate the security tab within your settings application on your emulated device and set up a fingerprint + password. First, select an arbitrary password that you will remember. Then, it will prompt you to enter your fingerprint. Navigate to the 3-bullet drop down button for 'Extended controls' directly above your emulated device in your emulator panel. If you do not see a toolbar with this option, click the settings cog in the emulator panel and enable 'Show toolbar'. There will be a tab that says 'Fingerprint'. Click

this and you will find a drop down menu for which fingerprint to use along with a button that says ‘Touch sensor’. Remain on ‘fingerprint 1’ and press the button to set up your fingerprint.

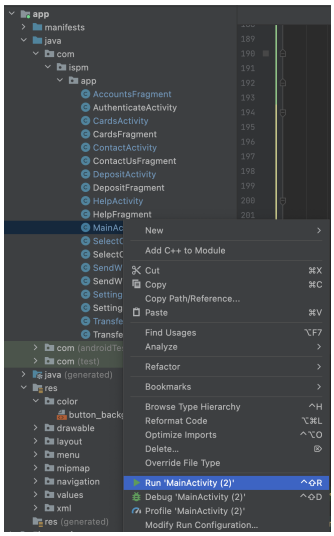

Figure 1: Example of how to find and run ’MainActivity.java’

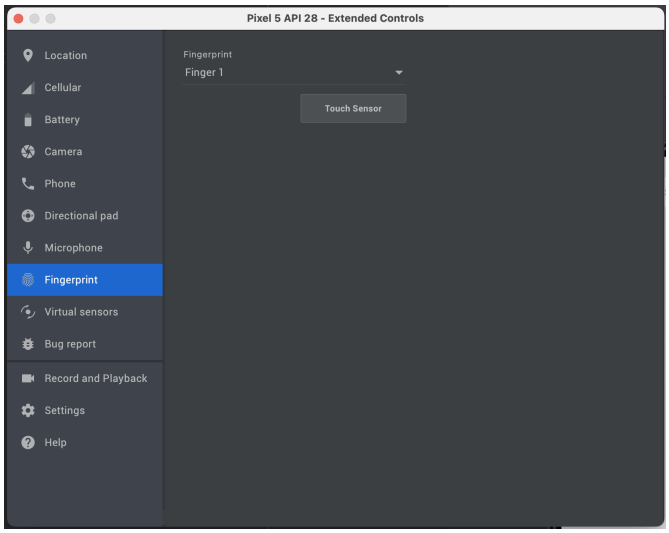

Figure 2: Example of fingerprint sensor window

Task Set 1: Creating the Attack Application

Task 1.1: Exploring the Vulnerabilities: Intent Flags

Here we will cover launching an activity via View Intents and set you up for how to perform an injection. In order for us to enlist the race condition within the Activity Lifecycle, the malicious activity must be launched within 100ms of the target activity from starting. Beneath your *raceattack* module, in your *raceattack/MainActivity.java* file, you will notice you are provided a skeleton application with empty functions. For example, you will have an event listener bound to the button enumerated in your *covering.xml* file (found from *res -> layout* beneath your project structure). Once the button is clicked, it calls the *launchAttack()* method. In this *launchAttack()* method, you should create a launch intent for our target application. Take note of the code example below.

```
getPackageManager().getLaunchIntentForPackage(victimPkgName);
```

Find the package name of the target banking application, create a launch intent using it, and store it within a variable to use later. Once you have done this, it is now time to set flags for the launch intent. If you navigate to the documentation for Intents, you can find its properties below; many of which are flags. Flags allow you to control the specific behavior of an Intent and are critical for implementing this attack. The few flags we will be using for our implementation are:

- *Intent.FLAG_ACTIVITY_NO_ANIMATION*

  This flag will prevent the system from applying an activity transition animation to go to the next activity state.

- *Intent.FLAG_ACTIVITY_NEW_TASK*

  This flag makes an activity become the start of a new task on this history stack. It is typically used for launcher style applications.

- *Intent.FLAG_ACTIVITY_MULTIPLE_TASK*

This flag is used to create a new task and launch an activity into it.

Flags are just integer codes that tell the program to enable or disable certain features. For us to set the flags, we will call the *setFlags()* method using the dot operator on our previously created Intent variable. We use the bitwise OR operator '|' to set multiple flags to one Intent in a single call. Go ahead and set each one of the above flags on our intent. Once you have done this, simply call the function *startActivity()* and pass your Intent as an argument. Now, if you run your application and click the button, you should notice the victim application is launched followed by your malicious activity. Reference the figure 3 on page 7.

Action

1.1.1 - Create a launch intent for the victim application, set the necessary flags, and submit a screenshot of your *launchAttack()* function.
1.1.2 - Create a launch intent for the covering activity, set the necessary flags, and submit a screenshot of your *launchCover()* function.

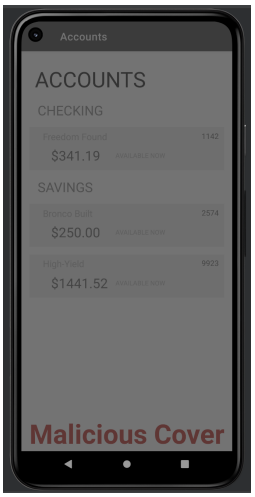

Figure 3: Example of malicious overlay

Task 1.2: Input Listener: Necessary Components (Touch Modals)

Now that your malicious app is launched over the victim application, you will now explore how to prevent the physical input listener from pausing when covered with your malicious activity. This section very similarly models the previous one; we will set flags for your malicious activity rather than the launch intent towards the victim application. Navigate to your *CoveringActivity.java* file beneath the *raceattack* module. The *CoveringActivity.java* represents the activity that is launched over the victim application while *MainActivity.java* is simply the handling application. Since this is not a launch intent, we end up setting flags for WindowManager.LayoutParams, which as the name explains, is the window manager for our activity. The functionality is very similar, minus a few minor tweaks. First and foremost, the two flags we will be setting for our window are:

- *WindowManager.LayoutParams.FLAG_NOT_TOUCH_MODAL*

  This flag allows any pointer events outside of the window to be sent to the windows behind it.

- *WindowManager.LayoutParams.FLAG_NOT_TOUCHABLE*

  This flag leaves the touch (or ignores it) to be handled by some window below this window (in Z order).

In order to set these flags to the entire activity, call the function *getWindow()* and call the *addFlags()* method using the dot operator behind it. The reason we call *addFlags()* is because it *appends* the flag codes rather than replaces them like *setFlags()* does.

Action

1.2.1 - Set the flags for your covering activity and submit a screenshot of your code.

Task Set 2: Creating the Overlay

Task 2.1: Creating Instance Variable

Now that we have a working overlay, we must be able to determine what to place on the screen in order to trick the user into correctly navigating to our target destination. Though the intention of the overlay is to obscure the view of the application beneath, we can determine when the application changes screens based on the triggering of the *onPause()* lifecycle function.

To do so, we will create a global instance variable, wrapped in its own class, in order to keep track of each time we navigate to a different section of the application. Right click the *raceattack* folder (not the module) containing all of your classes, and select 'New > Java Class'. You can name this arbitrarily. Create an instance variable *screenCount* that is equal to 0, as well as an object of this class within the class's instance variables.

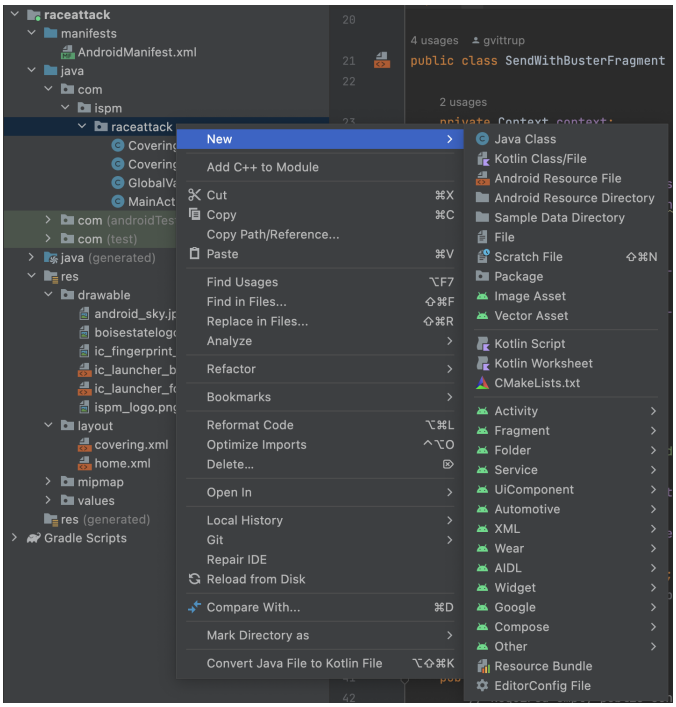

Figure 4: Example of malicious overlay

Now create a static public method that returns your class type called *getInstance()*. This method should check to see if your previously created class object is null; if so, it sets instance to a new object, otherwise it simply returns the existing object.

Navigate back to your *CoveringActivity.java* class. You will notice that one of your provided functions is an overridden *onPause()*. This is where you will keep track of the screen count by simply calling your instanced variable *screenCount*. To validate this works, follow the example code below to print to the console to ensure that it is working.

```
Log.d("screenCount: ", GlobalVariable.getInstance().screenCount);
```

If all is working, you may proceed to the following section.

Action

2.1.1 - Create a class and a global variable to keep track of the number of screen transitions, and submit a screenshot of your code.
2.1.2 - Update the *onPause()* function in *CoveringActivity.java* to increment and log the *screenCount* variable.

Task 2.2: Creating Overlay

In your project window on the left-hand side, navigate to your 'res' package, and open the folder 'layout'. Beneath this, you will find your *covering.xml* file. Once you open this, you will notice in the top right corner of your viewport that you have the option to change between your code and the design viewport. Here is where you will implement any respective buttons, input fields, or images to match the log-in page from the target application as much as possible. You will notice that you have a skeleton outline for a few primitive operations to get you started on utilizing these features, but any additional features or solutions can be found on the Android Studio UI documentation.

Now that you have a malicious application set up, it is your job to create a series of prompts that tricks the user into navigating through the application. If you explore the victim application, you will notice that it takes on the form of a student banking application. The app is kept rudimentary and only certain buttons contain functionality. The proper path to take is *Transfer & Pay -> Send with Buster -> Malicious Mallory (the greyed out option)*, and then from there you are to select an account and corresponding amount of money to take from the user. Once you click 'Send with Buster', the application will load you to a fingerprint authorization screen, at which point you will encourage the user to enter their fingerprint. You will notice that your *covering.xml* file contains some basic properties to help you get started, but it is up to you to create the entire overlay.

You will need to create a separate overlay for each part of the application you navigate to and change whats visible each time the screen changes. The encouraged way to implement this is by creating a section for each screen and giving it an id. Once you have done this, you will be able to toggle the visibility of each screen based on the count of your *screenCount* instance variable that you created in the previous step. Create a switch case within your *CoveringActivity.java* class based on *screenCount* and toggle the visibility for each respective layout based on that. You can do this by mimicing the code below:

```
findViewById(R.id.layout1).setVisibility(View.GONE);
```

It might be preferrable to store each individual layout in a variable.

Because we can not determine what screen is being viewed on the application below from a zero-permission standpoint, the design is very fickle. Your overlay will only work if you successfully deceive the user into following your prompts, and otherwise will not work as intended. You are encouraged to use images, animations, timers, and any other method that will increase the effectiveness of your overlay. Create an overlay up until the final page for authorization.

Action

2.2.1 - Create your navigational overlay and submit 3 screenshots from different sections of the page.

Lastly, once you reach the authorization screen, you will notice that if you do not use the correct fingerprint or cancel the input listener, the authorization will fail. Test yourself by utilizing fingerprint 1 and fingerprint 2 on the page. For this section, the most practical way to trick a user into inputting their fingerprint is by mimicking a lock-screen.

Action

2.2.2 - Create a fake lock-screen for the final destination of your overlay and submit a screenshot.

Task Set 3: Game Theory

Video: *Semi-Separating Equilibrium / Partially-Pooling Equilibrium by Willian Spaniel on YouTube*

Here in our last task set, we are going to explore a context and related game theory tree that will help us understand adversarial thinking between a developer and an attacker.

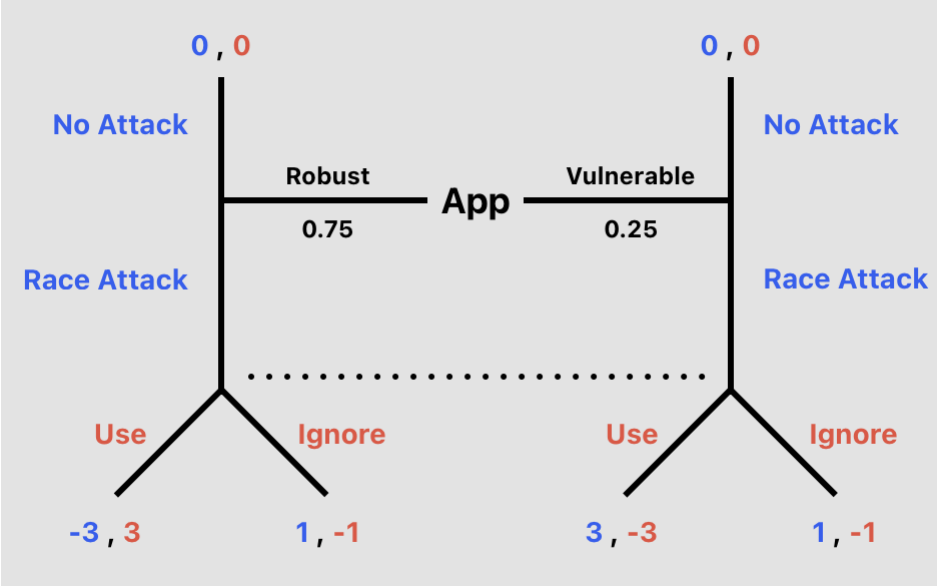

Figure 5: Game theory tree

Through this game theory tree, our intention will be to find the pooling equilibrium and each players mixed strategy. The context goes as follows: An app has a 75% chance of being robust, and a 25% chance of being vulnerable. Player 1 (the attacker, colored in blue) decides whether or not to attack the user. The attack would be similar to our implementation, such as prompting the user to input physical touch. Player 2 (colored in red) is the user, and can decide to use (or follow) the prompt given by the attacker, or can simply ignore it. The thing is - if the user follows the attackers prompt while the app is robust, it will trigger a security flag and the attack will completely fall apart. However, if the user ignores it, the user fails to identify a malicious app on their device and provides player 1

the option of monitoring state information, thus still losing overall. If player 1 attacks a vulnerable app, regardless of player 2's choice, player 1 will profit. Therefore, player 1 will always choose to attack.

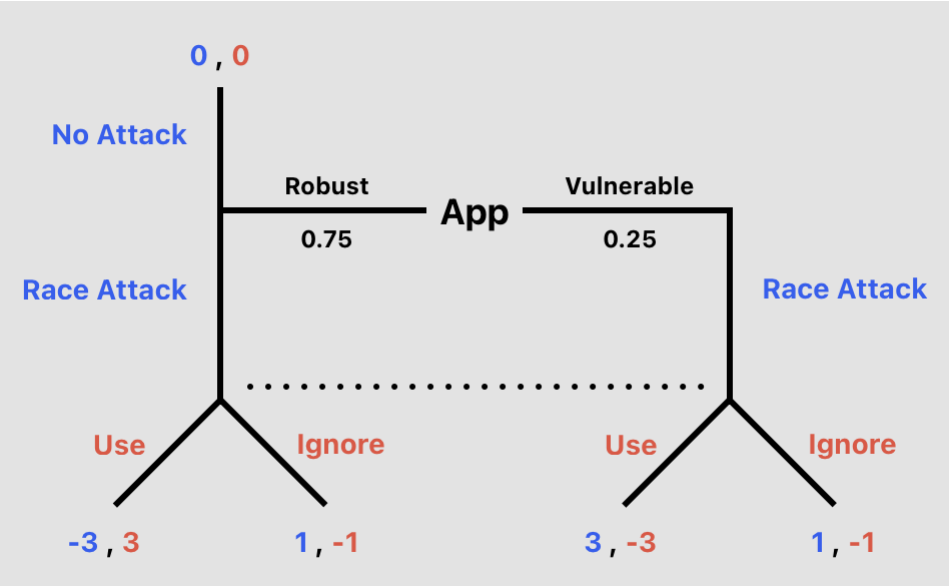

Figure 6: Player 1's profitable deviation

This allows us to remove this section of the game theory tree, as we have identified that player 1 (who makes the first move) will never choose to not attack when the app is vulnerable. Now let's consider player 2 to be an informed developer who is trying to test the security of their device and the API's that it runs on. Let's calculate player 2's pooling strategy. Use the following formula below, where R signifies the probability that an app is robust, and V signifies the probability that the app is vulnerable:

$$u(p2\_ignore) = R(p2\_ignore) + V(p2\_ignore)$$
$$u(p2\_use) = R(p2\_use) + V(p2\_use)$$

Given player 2's pooling strategy, does player 1 have a profitable deviation? It does, because now the bluff of attacking a seemingly robust app backfires, because player 1 could stick with 0 for not attacking compared

to the expected -3 if they did attack. This means we can not find any equilibria given the current information. The strategy must mix. In our game, its impossible to play a pure strategy, therefore it must mix.

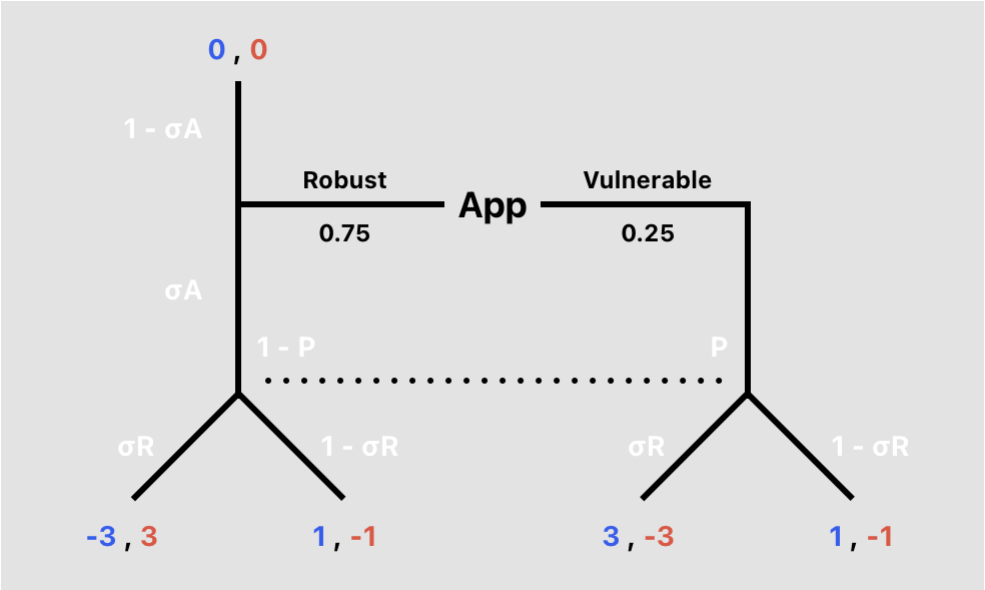

Figure 7: Mixed strategy tree

Using this tree, we will calculate player 2's mixed strategy. To make an attack on a robust app indifferent, player 2 must mix between using and ignoring the prompt. We must calculate the utility based on player 1 attacking and their expected utility from each decision. Using indifference conditions, we can do this with the following formula:

$$u(p1\_attack) = \sigma_R(p1\_use) + (1 - \sigma_R)(p1\_ignore)$$

We only calculate the probability for player 1's attack because if they do not attack, both players receive a utility of 0. If you work out the math, you will be left with the value of σ_R, which equals the probability that player 2 should use the app and is player 2's equilibrium strategy. But this outcome only benefits player 2 when the app is robust, therefore we need to be able to calculate player 2's belief that the app is vulnerable. Player 2's belief

is represented by the variable P, and can be calculated by the following formula:

$$u(p2\_ignore) = P(p2\_ignore) + (1 - P)(p2\_ignore)$$
$$u(p2\_use) = P(p2\_use) + (1 - P)(p2\_use)$$

Work out the math and find the value for the variable P. This represents the portion of the time that player 2's belief is that they are facing an attack against a vulnerable app. From here, the last thing we need to do is use Bayes' rule in a non-trivial way for us to find a perfect Bayesian equilibrium. We are looking for the robust types mixed strategy that results in a value exactly equal to P found in the previous step. This can be done with the following formula:

$$P = \frac{(0.25)(1)}{(0.25)(1) + (0.75)(\sigma_A)}$$

Add in your value for P and solve for σ_A which represents player 1's probability that the app is robust and they attack.

To recap:

- Player 1 attacks a robust app with a probability of σ_A
- After observing player 1 attack, player 2 believes that the app is vulnerable with a probability of P
- Player 2 then uses the app with a probability of σ_R

Action

3.1.1 - Calculate player 2's pooling equilibrium; which strategy profitably deviates?
3.1.2 - Calculate the value of σ_R, which represents player 2's mixed equilibrium strategy. What is the probability that player 2 uses the app?
3.1.3 - Calculate the value of P, which represents player 2's belief that they are facing a vulnerable app. What is this probability?
3.1.4 - Calculate the value of σ_A, which represents the probability that the app is robust and player 1 chooses to attack.

Submission

Submit a lab report utilizing the provided template to your instructor, which should include screenshots, thoughts, challenges, and lessons learned from the lab. Furthermore, zip and attach your project files with the relevant java files for submission.

Summary

Many mobile apps use these physical interfaces to receive user-input for authentication/authorization operations including one-click login, fingerprint-based payment approval, and face/voice unlocking. Our focus on this lab was to mislead an unsuspecting user with a zero-permission malicious application to feed screen-touch and fingerprint input in order to unintentionally authorize a fund-transfer. Furthermore, we explored some conventional design patterns that led to increased security risks regarding these types of attacks.

Trouble Shooting

Refactoring the Project Name

• Make sure you change the respective lines within build.gradle and AndroidManifest.xml to match what you refactored your project to. Then, select File->Invalidate Caches->Restart, and your project should rebuild with the correct dependencies. If this does not work, you can select Build->Clean Project and Build->Rebuild Project and then retry invalidating the caches. This will fully correct your problem.

Images: 'Trying to render too large of a bitmap' error

• Scroll down to your 'res' folder, right click, and create new Android Resource Folder. Select the dropdown and click 'Drawable'. In the navigator pane on the bottom left, scroll down and select 'Density'. This will create a 'Drawable-xxxhdpi' folder. Now, navigate to your drawable in the 'Resource Manager', right click, and move the file into that newly created folder. If this does not work, delete your image, reupload it, and before importing it change its version to 'UI Mode'. Repeat the previous process and everything should work as intended.

References

[1] Xianbo Wang, Shangcheng Shi, Yikang Chen, and Wing Cheong Lau. Phyjacking: Physical input hijacking for zero-permission authorization attacks on android. In *NDSS*, 2022.